

Zero to Hero Study Handbook: AI Job Market Copilot (Ollama)

This handbook is built from static analysis of the repository code and artifacts.

Module 1: Foundations & Architecture

1) What this project does

This repository implements an end-to-end AI/ML remote jobs pipeline:

1. Fetch live job postings from two sources:
 - Remote API (`src/job_market_copilot/clients/remotive.py`)
 - We Work Remotely RSS (`src/job_market_copilot/clients/wwr.py`)
2. Normalize and deduplicate records into a canonical schema (`src/job_market_copilot/processing/normalize.py`).
3. Score AI relevance with rule-based logic (`src/job_market_copilot/processing/relevance.py`).
4. Enrich AI candidates with local LLM metadata using Ollama, with optional Hugging Face fallback, then heuristic fallback (`src/job_market_copilot/processing/enrich.py`).
5. Produce analytics, charts, tables, and a markdown report (`src/job_market_copilot/analysis/metrics.py`, `src/job_market_copilot/reporting/report.py`).
6. Visualize outputs in Streamlit (`app/streamlit_app.py`).

Main use cases:

- Learning a realistic data + LLM pipeline (not toy CSV workflows).
- Running repeatable market snapshots from live feeds.
- Demonstrating a portfolio-quality applied AI project with CLI, tests, and dashboard.

2) Core paradigms and patterns used in this repo

Definitions first, then where they appear:

- Async I/O pipeline stage: Definition: network-bound tasks run concurrently using `asyncio`. In repo: `run_ingestion()` uses `await asyncio.gather(remotive.fetch_remotive_jobs(...), wwr.fetch_wwr_jobs(...))`.
- Functional dataframe transformations: Definition: data is transformed via declarative column operations rather than mutable row-by-row state. In repo: Polars transformations in `normalize.py`, `relevance.py`, `metrics.py`.
- Orchestrated staged pipeline: Definition: the workflow is decomposed into explicit stages with clear artifacts between stages. In repo: `run_ingestion -> run_relevance_filter -> run_enrichment -> run_analytics` in `pipeline.py`.

- Rule-based classifier before LLM: Definition: deterministic heuristics filter/scope data before expensive model calls. In repo: `score_relevance()` creates `ai_relevance_score` and `is_ai_role_rule`.
- Failover chain for model enrichment: Definition: if primary model path fails, system degrades gracefully through fallback paths. In repo: `JobEnricher._enrich_row()` tries Ollama -> HF fallback (optional) -> heuristic.
- Typed configuration via environment: Definition: runtime settings are centralized in a typed model with defaults and env overrides. In repo: `Settings(BaseSettings)` in `config.py`.
- Artifact-driven application layer: Definition: UI and reports consume generated artifact files instead of re-running heavy logic. In repo: Streamlit app reads `artifacts/metrics.json` and `artifacts/ai_jobs_enriched.parquet`.

3) Architecture: components and interactions

Key components:

- CLI layer: `src/job_market_copilot/cli.py`
- Pipeline orchestrator: `src/job_market_copilot/pipeline.py`
- Source clients: `src/job_market_copilot/clients/remotive.py`, `src/job_market_copilot/clients/wvr.py`
- Processing layer: normalization, relevance scoring, enrichment (`src/job_market_copilot/processing/*`)
- Analytics/reporting: `src/job_market_copilot/analysis/metrics.py`, `src/job_market_copilot/reporting/report.py`
- UI layer: `app/streamlit_app.py`
- Settings + logging: `src/job_market_copilot/config.py`, `src/job_market_copilot/logging_utils.py`

Main flow (ASCII diagram):

```

CLI: job-market-copilot (Typer)
|
| run-all
v
pipeline.run_all(settings)
|
+--> run_ingestion(settings)
|     +--> remotive.fetch_remotive_jobs()
|     +--> wvr.fetch_wvr_jobs()
|     +--> normalize_records()
|     +--> deduplicate_records()
|     \--> artifacts/jobs_normalized.parquet
|
+--> run_relevance_filter(df, settings)
|     +--> score_relevance(threshold=settings.ai_keyword_threshold)

```

```

|         +--> artifacts/jobs_scored.parquet
|         \--> artifacts/jobs_ai_candidates.parquet
|
+--> run_enrichment(ai_df, settings)
|     +--> JobEnricher.enrich_dataframe()
|     |     +--> _try_ollama()
|     |     +--> _try_hf() [optional]
|     |     \--> _heuristic_fallback()
|     \--> artifacts/ai_jobs_enriched.parquet
|
\--> run_analytics(df_all, df_ai, settings)
    +--> compute_metrics()
    +--> save_metrics() -> artifacts/metrics.json
    +--> save_charts() -> artifacts/charts/*.html
    +--> render_report() -> artifacts/reports/job_market_report.md
    \--> artifacts/tables/*.csv

```

Streamlit App

app/streamlit_app.py reads artifacts and renders dashboard

Module 2: Repository Map

Focus files a new contributor should understand first:

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
pyproject.toml	Project metadata, dependencies, CLI script entrypoint	project.scripts.job_market_copilot = "job_market_copilot.py:cli_app"	dependencies, dependency list, pythonpath
.env.example	Environment variable template	N/A	config ENABLE_HF_FALLBACK, HF_TOKEN, OLLAMA_BASE_URL, OLLAMA_MODEL, OLLAMA_QUALITY_MODEL, OLLAMA_TEMPERATURE, MAX_JOBS_PER_SOURCE, MAX_JOBS_FOR_ENRICHMENT, AI_KEYWORD_THRESHOLD
README.md	User-facing quick start and architecture summary	N/A	Canonical command sequence (uv sync, CLI commands)

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
src/job_market_configs/ src/job_market_configs/ config.py	Typical/config.py settings and directory resolution	Settings, get_settings, resolve_path, ensure_directories	All runtime knobs (timeouts, retries, model names, thresholds, paths)
src/job_market_configs/ src/job_market_configs/ CLI.py	Typical/CLI.py commands and dispatch to pipeline	app, ingest_cmd, enrich_cmd, analyze_cmd, run_all_cmd, serve_app_cmd	CLI precondition checks for required artifacts
src/job_market_configs/ src/job_market_configs/ pipeline.py	Pipeline/pipeline.py orchestration	run_ingestion, run_relevance_filters, run_enrichment, run_analytics, run_all	Uses settings.ai_keyword_threshold, settings.ollama_model, artifact paths
src/job_market_configs/ src/job_market_configs/ remote.py	Remote/remote.py fetch + raw payload persistence	fetcher.py, fetch_remotive_jobs, save_raw_payload	REMOTE_URL, request timeout, retries, user agent, max per source
src/job_market_configs/ src/job_market_configs/ WWR/RSS feeds/wwr.py	WWR/RSS feeds/wwr.py parse, HTML stripping, save payload	fetch_text, _parse_item, _strip_html, fetch_wwr_jobs, save_raw_payload	WWR_RSS_URL, same timeout/retry settings
src/job_market_configs/ src/job_market_configs/ parser/processing/normalize.py	Parser/processing/normalize.py mapping + dedup logic	normalize_data, _canonical_url, normalize_records, deduplicate_records	Dedup key strategy: canonical URL
src/job_market_configs/ src/job_market_configs/ rules/predAI/relevance.py	Rules/predAI/relevance.py relevance scoring	relevance, _count_matches	STRONG_AI_KEYWORDS, SECONDARY_AI_KEYWORDS, exclusions, threshold
src/job_market_configs/ src/job_market_configs/ rules/processing/enricher.py	Rules/processing/enricher.py enrichment with robust fallback chain	Enricher, EnrichmentOutput, enrich_dataframe, _enrich_row, _try_ollama, _try_hf, _heuristic_fallback	_ALLOWED_ROLE_FAMILIES, _ALLOWED_SENIORITY, _SKILL_LEXICON, model/fallback settings

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
src/job_market_copilot/analysis/metrics.py	Analysis metrics, chart generation	compute_metrics, save_metrics, save_charts, _explode_list_column	Output schema in metrics.json, chart filenames
src/job_market_copilot/reporting/reporter.py	Model/reporting rendering	render_report, _REPORT_TEMPLATE	Report template fields and rendering timestamp
app/streamlit_app.py	Dashboard over generated artifacts	top-level Streamlit flow	PROJECT_ROOT, ARTIFACTS, filters for source and seniority
scripts/run_pipeline.py	Notepad-friendly convenience runner	main()	Equivalent to end-to-end pipeline trigger
src/job_market_copilot/schema.py	Pydantic schema definitions	CanonicalJob, EnrichmentResult	Canonical schema field names and types
tests/test_normalize.py	Validates normalization + dedup behavior	test_normalize_and_deduplicate	Ensures duplicate records URL variants collapse
tests/test_relevance.py	Validates relevance scoring behavior	test_relevance_scoring	Ensures AI jobs identification logic
tests/test_enrich_fallback.py	Validates heuristic fallback behavior	test_heuristic_fallback	Ensures fallback assigns valid role/seniority/comp signal
notebooks/*.ipynb	Learning sequence for pipeline stages	Notebook stage progression	01 ingest, 02 normalize, 03 enrich, 04 report

Module 3: Core Execution Flows

Flow A: Full pipeline (run-all) from CLI to artifacts

Entrypoint:

```
# src/job_market_copilot/cli.py
@app.command("run-all")
def run_all_cmd() -> None:
```

```
settings = get_settings()
metrics = asyncio.run(run_all(settings))
```

Step-by-step:

1. `job-market-copilot run-all` starts Typer app (`cli.py`).
2. `@app.callback()` runs `configure_logging()` from `logging_utils.py`.
3. `run_all_cmd()` builds `Settings` via `get_settings()`.
4. `pipeline.run_all(settings)` executes:
 - `run_ingestion(settings)`
 - `run_relevance_filter(df_all, settings)`
 - `run_enrichment(ai_candidates, settings)`
 - `run_analytics(df_all, enriched, settings)`
5. Final log reports `total_jobs`, `ai_jobs`, `ai_share`.

Flow B: Ingestion internals

Source code path: `pipeline.run_ingestion()`

1. Creates required directories with `settings.ensure_directories()`.
2. Concurrently fetches two sources:
 - `remoteive.fetch_remoteive_jobs(settings)` (JSON API)
 - `wwr.fetch_wwr_jobs(settings)` (RSS parsed to dicts)
3. Saves raw payloads:
 - `data/raw/remoteive_jobs.json`
 - `data/raw/wwr_jobs.json`
4. Normalizes records into one schema with `normalize_records(remoteive_jobs, wwr_jobs)`.
5. Deduplicates with `deduplicate_records(normalized)`.
6. Writes `artifacts/jobs_normalized.parquet`.

Flow C: Relevance scoring and AI candidate filtering

Source code path: `pipeline.run_relevance_filter()`

1. Calls `score_relevance(df, threshold=settings.ai_keyword_threshold)`.
2. `score_relevance()` computes:
 - `ai_relevance_score` (int)
 - `is_ai_role_rule` (bool)
3. Writes full scored output to `artifacts/jobs_scored.parquet`.
4. Filters rows where `is_ai_role_rule == True`.
5. Writes candidates to `artifacts/jobs_ai_candidates.parquet`.

Flow D: Enrichment with model failover

Source code path: `pipeline.run_enrichment()` and `processing/enrich.py`

1. Instantiates `JobEnricher(settings)`.
2. `enrich_dataframe()` limits rows to `max_jobs_for_enrichment`.

3. For each row:
 - build prompt (`_build_prompt`)
 - try Ollama JSON output (`_try_ollama`)
 - validate payload (`_validate_payload`)
 - if invalid/unavailable and enabled, try HF (`_try_hf`)
 - else use `_heuristic_fallback`
 4. If enrichment limit truncates input:
 - tail rows are preserved with defaults:
 - `role_family="other"`
 - `seniority="unknown"`
 - `summary="Not enriched due to configured max_jobs_for_enrichment limit."`
 - `model_source="fallback_limit"`
 5. Writes `artifacts/ai_jobs_enriched.parquet`.
- Expected model JSON keys (enforced in prompt and validator):
- `role_family`
 - `seniority`
 - `core_skills`
 - `tooling`
 - `summary`
 - `compensation_signal`

Flow E: Analytics + report generation

Source code path: `pipeline.run_analytics()`

1. Computes aggregate metrics via `compute_metrics(df_all, df_ai)`.
2. Writes `artifacts/metrics.json` via `save_metrics()`.
3. Writes charts via `save_charts()`:
 - `artifacts/charts/top_skills.html`
 - `artifacts/charts/role_family_mix.html`
 - `artifacts/charts/ai_roles_by_source.html`
4. Renders markdown report to:
 - `artifacts/reports/job_market_report.md`
5. Exports CSV table previews:
 - `artifacts/tables/ai_jobs_preview.csv`
 - (normalized preview already present in repo) `artifacts/tables/normalized_preview.csv`

Flow F: Streamlit dashboard serving and rendering

Entrypoint chain:

1. CLI command `job-market-copilot serve-app --port 8501 (cli.py)`.
2. Executes subprocess:
 - `python -m streamlit run app/streamlit_app.py --server.port <port>`

3. Streamlit app:

- validates artifacts existence
- reads `metrics.json` + `ai_jobs_enriched.parquet`
- renders KPI cards, source/seniority filters, role pie chart, skill bar chart, job explorer table

Key input/output data shapes (from code + artifacts)

A) Raw source dicts consumed by normalizer

- Remote fields used:
 - `id`, `title`, `company_name`, `url`, `publication_date`
 - `candidate_required_location`, `category`, `description`
 - `salary`, `job_type`, `tags`
- WWR parsed fields produced by `_parse_item`:
 - `id`, `company`, `title`, `url`, `published`
 - `location`, `category`, `employment_type`, `description`, `tags`

B) Normalized dataframe columns Produced by `normalize_records()`:

- `source_job_id`
- `source`
- `title`
- `company`
- `url`
- `canonical_url`
- `published_at`
- `location`
- `category`
- `raw_description`
- `salary_text`
- `employment_type`
- `tags`

After dedup:

- plus temporary dedup columns during processing (`dedup_key`, `published_rank`) with `published_rank` dropped before return.

C) Scored dataframe extra columns Added by `score_relevance()`:

- `ai_relevance_score` (Int64)
- `is_ai_role_rule` (bool)

D) Enriched dataframe extra columns Added by `JobEnricher.enrich_dataframe()`:

- `role_family` (str)
- `seniority` (str)

- `core_skills (list[str])`
- `tooling (list[str])`
- `summary (str)`
- `compensation_signal (str)`
- `model_source (str, e.g. ollama, hf_fallback, heuristic_fallback, fallback_limit)`

E) **Metrics output schema** `compute_metrics()` returns:

- `total_jobs (int)`
 - `ai_jobs (int)`
 - `ai_share (float)`
 - `salary_coverage_ai (float)`
 - `source_counts (list[{"source": str, "count": int}])`
 - `role_counts (list[{"role_family": str, "count": int}])`
 - `seniority_counts (list[{"seniority": str, "count": int}])`
 - `top_skills (list[{"core_skills": str, "count": int}])`
 - `top_tools (list[{"tooling": str, "count": int}])`
 - `top_companies (list[{"company": str, "count": int}])`
-

Module 4: Setup & Run Guide

1) Prerequisites

- OS: Linux/macOS/Windows with Python 3.12.10 support.
- Package/environment manager: `uv` (project standard).
- Local LLM runtime: Ollama reachable at `OLLAMA_BASE_URL` (default `http://127.0.0.1:11434`).
- Optional fallback: Hugging Face token if enabling fallback path.

2) Installation steps

From repository root:

```
uv python pin 3.12.10
uv sync --dev
```

3) Environment configuration

Create `.env` from template:

```
cp .env.example .env
```

Environment keys present in `.env.example`:

- `ENABLE_HF_FALLBACK`
- `HF_TOKEN`
- `OLLAMA_BASE_URL`

- OLLAMA_MODEL
- OLLAMA_QUALITY_MODEL
- OLLAMA_TEMPERATURE
- MAX_JOBS_PER_SOURCE
- MAX_JOBS_FOR_ENRICHMENT
- AI_KEYWORD_THRESHOLD

Other configurable settings exposed in `Settings` (`config.py`) and loadable via environment:

- Path and runtime keys:
 - PROJECT_ROOT, DATA_DIR, RAW_DIR, ARTIFACTS_DIR
 - USER_AGENT, REQUEST_TIMEOUT_SECONDS, REQUEST_RETRIES
 - HF_MODEL

4) Typical command sequences

Full end-to-end run:

```
uv run job-market-copilot run-all
```

Stage-by-stage run:

```
uv run job-market-copilot ingest
uv run job-market-copilot enrich
uv run job-market-copilot analyze
```

Serve dashboard:

```
uv run job-market-copilot serve-app --port 8501
```

Notebook-friendly script:

```
uv run python scripts/run_pipeline.py
```

5) Artifact locations after run

- artifacts/jobs_normalized.parquet
- artifacts/jobs_scored.parquet
- artifacts/jobs_ai_candidates.parquet
- artifacts/ai_jobs_enriched.parquet
- artifacts/metrics.json
- artifacts/charts/*.html
- artifacts/reports/job_market_report.md
- artifacts/tables/*.csv
- data/raw/remotive_jobs.json
- data/raw/wvr_jobs.json

6) Migration/seeding requirements

This project has no database migrations or seed scripts.

- No ORM migration files.
- No database bootstrap phase.
- Persistence is file-based (JSON, Parquet, CSV, Markdown, HTML artifacts).

7) Export handbook to PDF

From repo root:

```
pandoc ZERO_TO_HERO_STUDY_HANDBOOK.md -o ZERO_TO_HERO_STUDY_HANDBOOK.pdf
```

Module 5: Study Plan & Practice Exercises

1) Ordered study plan for a new learner

1. Read `README.md` to understand the project goal and outputs.
2. Read `pyproject.toml` to see dependencies and true CLI entrypoint.
3. Read `src/job_market_copilot/config.py` to learn all runtime knobs.
4. Read `src/job_market_copilot/cli.py` to map user commands to internal functions.
5. Read `src/job_market_copilot/pipeline.py` to get the full orchestration view.
6. Read source clients: `clients/remotive.py`, `clients/wwr.py`.
7. Read normalization + dedup: `processing/normalize.py`.
8. Read relevance logic: `processing/relevance.py`.
9. Read enrichment logic deeply: `processing/enrich.py` (largest behavior surface).
10. Read analytics/reporting: `analysis/metrics.py`, `reporting/report.py`.
11. Read UI layer: `app/streamlit_app.py`.
12. Read tests in `tests/` to validate your understanding of intended behavior.

2) Practice exercises (with solution outlines)

Exercise 1: Trace the run-all flow Task: Starting at CLI command `run-all`, list all stage functions called in order and one output file from each stage.

Solution outline:

1. `run_all_cmd()` in `cli.py` calls `pipeline.run_all(settings)`.
2. `run_ingestion()` -> `artifacts/jobs_normalized.parquet`.
3. `run_relevance_filter()` -> `artifacts/jobs_scored.parquet` and `artifacts/jobs_ai_candidates.parquet`.
4. `run_enrichment()` -> `artifacts/ai_jobs_enriched.parquet`.
5. `run_analytics()` -> `artifacts/metrics.json`, charts, report, tables.

Exercise 2: Explain dedup strategy Task: Describe exactly how dedup keys are computed in `deduplicate_records()` and how tie-breaking works.

Solution outline:

1. If `canonical_url` has characters, dedup key is `canonical_url`.
2. Otherwise dedup key is lowercase `"title|company"`.
3. Rows are sorted by `dedup_key` and descending `published_rank`.
4. `unique(subset=["dedup_key"], keep="first")` keeps newest per key.

Exercise 3: Relevance scoring mechanics Task: For one row, list all signal buckets that can increase score and one condition that hard-zeros score.

Solution outline:

1. Score buckets: title patterns, strong keywords in title/category/tags/description, secondary keywords in title/category/description.
2. Hard-zero condition: title matches any regex in `NON_AI_TITLE_EXCLUSIONS` (e.g., customer support).

Exercise 4: Fallback chain reasoning Task: Document the exact enrichment fallback chain and when `model_source` becomes `hf_fallback` or `heuristic_fallback`.

Solution outline:

1. First try `_try_ollama()`.
2. If Ollama fails/invalid payload and `enable_hf_fallback` plus `hf_token` are set, try `_try_hf()`.
3. If HF works, validator output is returned with `model_source` overwritten to `hf_fallback`.
4. Otherwise `_heuristic_fallback()` returns `model_source="heuristic_fallback"`.

Exercise 5: Understand enrichment limit behavior Task: What happens when `df.height > max_jobs_for_enrichment`?

Solution outline:

1. Head subset is enriched.
2. Tail rows are preserved with synthetic defaults:
 - `role_family="other"`, `seniority="unknown"`, empty lists for `core_skills/tooling`,
 - summary note about limit,
 - `compensation_signal="unclear"`,
 - `model_source="fallback_limit"`.
3. Enriched head + fallback tail are concatenated vertically.

Exercise 6: Map metrics schema to dashboard usage Task: Which KPI cards in Streamlit map directly to `metrics.json` keys?

Solution outline:

1. Total Jobs -> `metrics["total_jobs"]`
2. AI/ML Jobs -> `metrics["ai_jobs"]`
3. AI Share -> `metrics["ai_share"]`
4. Salary Coverage -> `metrics["salary_coverage_ai"]`

Exercise 7: Understand command preconditions Task: Find which CLI commands enforce artifact existence before running and what they require.

Solution outline:

1. `enrich` requires `artifacts/jobs_normalized.parquet`.
2. `analyze` requires both `artifacts/jobs_scored.parquet` and `artifacts/ai_jobs_enriched.parquet`.
3. If missing, each raises `typer.BadParameter(...)`.

Exercise 8: Validate canonical schema mentally Task: Name at least 10 columns expected after normalization and identify which source-specific fields map into them.

Solution outline:

1. Columns include: `source_job_id`, `source`, `title`, `company`, `url`, `canonical_url`, `published_at`, `location`, `category`, `raw_description`, `salary_text`, `employment_type`, `tags`.
2. Example mappings:
 - Remote `company_name` -> `company`
 - Remote `publication_date` -> `published_at`
 - WWR `published` -> `published_at`
 - WWR `employment_type` -> `employment_type`

Exercise 9: Read the report template critically Task: Locate where the markdown report timestamp comes from and where the Ollama model string in the report comes from.

Solution outline:

1. Timestamp: `datetime.now().isoformat(timespec="seconds")` in `render_report()`.
2. Model string: passed from `pipeline.run_analytics()` as `settings.ollama_model`.

Exercise 10: Connect tests to implementation behavior Task: For each test file, state the behavioral guarantee it protects.

Solution outline:

1. `test_normalize.py`: duplicate cross-source records collapse correctly.
 2. `test_relevance.py`: AI-like row scores higher and is flagged true.
 3. `test_enrich_fallback.py`: heuristic fallback derives expected role/seniority/compensation signal.
-

Understanding Checklist

Use this before claiming you understand the repository:

- Can you explain the full call chain from `job-market-copilot run-all` to every artifact file generated?
- Can you explain why ingestion is async and which functions run concurrently?
- Can you describe the canonical normalized job schema and where each field originates?
- Can you explain dedup key construction and tie-breaking logic exactly?
- Can you explain `ai_relevance_score` and the final boolean gate `is_ai_role_rule`?
- Can you explain all three enrichment paths (Ollama, HF fallback, heuristic fallback)?
- Can you explain how `max_jobs_for_enrichment` changes output behavior?
- Can you explain how `compute_metrics()` builds `metrics.json` and how Streamlit consumes it?
- Can you run stage commands (`ingest`, `enrich`, `analyze`) and explain each precondition?
- Can you modify one scoring keyword list and predict downstream effects on artifacts?